

Assignment VI

AI Compute Platforms, Applications and Trends

Submission Include:

Student Name:

Student ID:

Assignment Title:

Question I : Git Version Control and File Restoration

You are developing a simple website and are required to manage different versions of the project using **Git**.

Perform the following tasks:

1. Create a directory named demo1. Inside the directory, create an index.html file containing:

This site is under construction!!

Initialize the directory as a Git repository. Check the repository status, stage the file, and create an appropriate initial commit.

2. Create a new file named login.html containing:

Welcome to the Login page!

Check the repository status, stage the new file, and commit the change with an appropriate commit message.

3. Modify index.html by replacing its existing content with:

Welcome to DAI Course!!

Stage and commit the modification.

4. Make the following changes to both files:

In index.html, add a second line:

Click here to Login!

In login.html, add a second line:

Enter your Username and Password below.

Stage both modified files and commit the changes.

5. Modify login.html by adding the following line:

The OTP option is coming soon! Update your mobile numbers!

Stage and commit the change.

6. Modify the OTP-related line in login.html so that it displays:

Enter the OTP here

Stage and commit the modification.

7. Users are experiencing problems with the OTP login functionality. The OTP-related changes must therefore be removed temporarily.

Using the **Git commit history**, identify the appropriate earlier version of login.html and restore the file so that it contains only:

Welcome to the Login page!

Enter your Username and Password below.

Do **not manually rewrite the file** to obtain the required version. Verify the restored contents and check the repository status.

Question II : Docker, Docker Hub and Kubernetes

Create a simple web application that displays:

Welcome to app1...version1

Perform the following tasks:

1. Create an index.html file containing the above message and create a Dockerfile using an **Nginx** base image to serve the web page.
2. Build a Docker image and tag it as:

<dockerhub-username>/app1:v1

Create a repository named app1 on Docker Hub and push the image to the repository

3. Using Minikube, create a Kubernetes Deployment named app1-deployment using the Docker image pushed to Docker Hub. Verify that the Deployment and Pod are running successfully.

4. Expose the application using a **NodePort Service** on port 80.
5. Obtain the URL of the Service and access the application from a web browser. Verify that the following message is displayed:

Welcome to app1...version1

Question III. GitHub, Docker and Jenkins

Create a GitHub repository named `jenkins-web-app`. Add an `index.html` file that displays “**Welcome to Jenkins Web Application – Version 1**” and create a `Dockerfile` using an Nginx image to serve the web page.

On the Ubuntu machine, clone the repository. Build a Docker image named `jenkins-web-app:v1` and run a container named `web-app` so that the application is accessible at:

`http://localhost:9000`

Create a **Jenkins Freestyle job** that retrieves the latest source code from the GitHub repository, builds the Docker image, removes/replaces the previously running container, and runs the updated application on port 9000.

Modify `index.html` in the GitHub repository to display “**Welcome to Jenkins Web Application – Version 2**”. Run the Jenkins job and verify that the updated application is available at:

`http://localhost:9000`

Demonstrate that Version 2 is displayed successfully.

Question IV. Flask, Docker Hub and Kubernetes Application Update

Create a simple **Flask web application** that displays your **Name** on the web page. Configure the Flask application to listen on `0.0.0.0` and run on **port 6000**.

Complete the following tasks:

1. Create the required Flask application and a `Dockerfile`. Build a Docker image for the application and tag it as:

`<dockerhub-username>/student-app:v1`

2. Create a Docker Hub repository named `student-app` and push the **Version 1** image to the repository.
3. Using Minikube, create a Kubernetes Deployment named `student-app` using the **Version 1** image. Verify that the Deployment is created and the Pod is running successfully.

4. Expose the Deployment using a **NodePort Service**, with the Service forwarding traffic to the Flask application running on **container port 6000**. Obtain the Service URL and verify that your **Name** is displayed in the browser.
5. Modify the Flask application so that the web page displays both your **Name and PRN Number**. Build a new Docker image and tag it as:

<dockerhub-username>/student-app:v2

Push the **Version 2** image to the **same Docker Hub repository**.

6. Without deleting or recreating the existing Kubernetes Deployment or Service, update the Deployment to use the **Version 2** image. Verify that the rollout is completed successfully. Refresh the **same Service URL** and demonstrate that both your **Name and PRN Number** are displayed.